Shannon Code

Source X produces a single symbol $x \in \{s_1, \ldots, s_m\}$ with prob. $p_1, \ldots, p_m$.

- Order symbols according to their probabilities. For simplicity we assume

$$p_1 \geq p_2 \geq p_3 \geq \cdots \geq p_m.$$

  (Otherwise we can simply relabel them.)
- To prepare the encoding, define for each $i \in \{1, \ldots, m\}$:

$$\ell_i := \left\lceil \log_2 \left(\frac{1}{p_i} \right) \right\rceil$$

$$F_i := \sum_{j=1}^{i-1} p_j \qquad \text{(with } F_1 = 0.\text{)}$$

- To encode $X = s_i$, express F_i in binary representation:

$$F_i = 0.\ b_i^{(1)}\ b_i^{(2)}\ b_i^{(3)}\ \ldots$$

  and obtain the codeword c_i as the first ℓ_i bits after the digit point:

$$\mathbf{c}_i = (b_i^1, b_i^{(2)}, \ldots, b_i^{(\ell_i)}).$$

Shannon Codes are Prefix-Free and Close to Optimal

Theorem

The Shannon code is prefix-free and has expected length $\mathsf{L}_{\text{Shannon}} = \sum_{i=1}^{m} p_i \ell_i < H(X) + 1$.

Proof.

Expected length has been proved in the first lecture.
We prove that code is prefix-free. Choose $i, i' \in \{1, \ldots, m\}$ with $i > i'$. By the ordering of the probabilities, $\ell_i \geq \ell_{i'}$ and it suffices to show that codeword $\mathbf{c}_{i'}$ is not a prefix of codeword $\mathbf{c}_i$. Proof is by contradiction. In fact, if $\mathbf{c}_{i'}$ is a prefix of $\mathbf{c}_i$, then they have same first $\ell_{i'}$ bits. By construction of the Shannon code and because $\tilde{F}_i > \tilde{F}_{i'}$ this means that

$$F_i - F_{i'} < 2^{-\ell_{i'}}.$$

But this contradicts the following inequality

$$F_i - F_{i'} = \sum_{j=i'}^{i-1} p_j \geq p_{i'} = 2^{-\log \frac{1}{p_{i'}}} \geq 2^{-\left\lceil \log \frac{1}{p_{i'}} \right\rceil} = 2^{-\ell_{i'}}.$$

□

Arithmetic Coding for a Single Source Symbol (Warmup!)

Source X produces a single symbol $x \in \{s_1, \ldots, s_m\}$ with prob. $p_1, \ldots, p_m$.

- Keep symbols unordered! (So p_i not necessarily smaller than p_{i-1}.)
- To prepare the encoding, define for each $i \in \{1, \ldots, m\}$:

$$\tilde{\ell}_i := \ell_i + 1 = \left\lceil \log_2 \left(\frac{1}{p_i} \right) \right\rceil + 1$$

$$\tilde{F}_i := F_i + 2^{-\tilde{\ell}_i} = \sum_{j=1}^{i-1} p_j + 2^{-\tilde{\ell}_i}$$

- To encode $X = s_i$, express $\tilde{F}_i$ in binary representation:

$$\tilde{F}_i = 0.\ b_i^{(1)}\ b_i^{(2)}\ b_i^{(3)}\ \ldots$$

and obtain the codeword $\mathbf{c}_i$ as the first $\tilde{\ell}_i$ bits after the digit point:

$$\mathbf{c}_i = \left(b_i^1, b_i^{(2)}, \ldots, b_i^{(\tilde{\ell}_i)} \right).$$

Arithmetic Codes are Prefix-Free and Almost Optimal I

Theorem

The Arithmetic code for a single source-symbol is prefix-free and has expected length $\mathsf{L}_{\text{Arithmetic}} = \sum_{i=1}^{m} p_i \ell_i < H(X) + 2$.

Proof:

Simply add $+1$ to the expected length of the Shannon code.
We prove that the code is prefix-free. Choose $i, i' \in \{1, \ldots, m\}$ with $i > i'$.
Since codewords are unordered, we have to show that $\mathbf{c}_{i'}$ is not a prefix of $\mathbf{c}_i$ *and* $\mathbf{c}_i$ is not a prefix of $\mathbf{c}_{i'}$. Notice that since $i > i'$ we have $\tilde{F}_i \geq \tilde{F}_{i'}$:

$$F_i - F_{i'} \geq p_{i'} - 2^{-\tilde{\ell}_{i'}} = 2^{-\log \frac{1}{p_{i'}}} - 2^{-(\lceil \log \frac{1}{p_{i'}} \rceil + 1)} \geq 0.$$

By the construction of the code, $\mathbf{c}_{i'}$ can only be a prefix of $\mathbf{c}_i$ if

$$\tilde{F}_i - \tilde{F}_{i'} < 2^{-\tilde{\ell}_{i'}} \tag{1}$$

and $\mathbf{c}_i$ can only be a prefix of $\mathbf{c}_{i'}$ if

$$\tilde{F}_i - \tilde{F}_{i'} < 2^{-\tilde{\ell}_i}. \tag{2}$$

Arithmetic Codes are Prefix-Free and Almost Optimal II

Proof continued:

But

$$p_{i'} = 2 \cdot 2^{-\left(\log \frac{1}{p_{i'}} + 1\right)} \geq 2 \cdot 2^{-\left\lceil \log \frac{1}{p_{i'}} \right\rceil + 1} = 2 \cdot 2^{-\tilde{\ell}_{i'}}$$

and therefore

$$\tilde{F}_i - \tilde{F}_{i'} = \sum_{j=i'}^{i-1} p_j + 2^{-\tilde{\ell}_i} - 2^{-\tilde{\ell}_{i'}} \geq p_{i'} + 2^{-\tilde{\ell}_i} - 2^{-\tilde{\ell}_{i'}} \geq 2^{-\tilde{\ell}_i} + 2^{-\tilde{\ell}_{i'}}.$$

By the nonnegativity of the exponential function, this last inequality contradicts both (1) and (2). □

Arithmetic Coding for Memoryless Source: Ideas

Memoryless source produces i.i.d. sequence $\boldsymbol{X}^n := (X_1, \ldots, X_n)$ where each X_t takes value in $\{1, \ldots, m\}$ with probabilities $p_1, \ldots, p_m$. Notice that the probability of each possible source sequence $\boldsymbol{x}^n = (x_1, \ldots, x_n)$ is $p_{\boldsymbol{x}^n} = \prod_{t=1}^n p_{x_t}$

The idea is to apply the previously proposed arithmetic coding for a single source symbol by viewing the sequence $\boldsymbol{X}^n$ as a single symbol of a supersource

Constructing the full prefix-free tree for this supersource is computationally too expensive. But all we want is that for any given source sequence $\boldsymbol{x}^n = (x_1, \ldots, x_n)$ we can efficiently compute the corresponding codeword, and from any given codeword we can efficiently recover the corresponding source sequence.

Arithmetic Coding for Memoryless Source: Encoding

- As the source symbols $x_1, x_2, \ldots, x_n$ arrive, recursively calculate

$$\begin{aligned} F_{\boldsymbol{x}^n} &:= \sum_{\boldsymbol{\xi}^n < \boldsymbol{x}^n} p_{\boldsymbol{\xi}^n} \\ &= \Pr[\boldsymbol{X}^n < \boldsymbol{x}^n] \\ &= \Pr\big[\boldsymbol{X}^{n-1} < \boldsymbol{x}^{n-1}\big] + \Pr\big[\boldsymbol{X}^{n-1} = \boldsymbol{x}^{n-1} \text{ and } X_n < x_n\big] \\ &= F_{\boldsymbol{x}^{n-1}} + p_{\boldsymbol{x}^{n-1}} \cdot F_{x_n}. \end{aligned}$$

- After having observed the entire source sequence, calculate

$$\tilde{\ell}_{\boldsymbol{x}^n} := \left\lceil \log\left(\frac{1}{p_{\boldsymbol{x}^n}}\right) \right\rceil + 1$$

$$\tilde{F}_{\boldsymbol{x}^n} := F_{\boldsymbol{x}^n} + 2^{-\tilde{\ell}_{\boldsymbol{x}^n}}$$

- Express $\tilde{F}_{\boldsymbol{x}^n}$ in binary representation:

$$\tilde{F}_{\boldsymbol{x}^n} = 0.\ b^{(1)}_{\boldsymbol{x}^n}\ b^{(2)}_{\boldsymbol{x}^n}\ b^{(3)}_{\boldsymbol{x}^n}\ \ldots$$

and obtain the codeword $\mathbf{c}_{\boldsymbol{x}^n}$ as the first $\tilde{\ell}_{\boldsymbol{x}^n}$ bits after the digit point:

$$\mathbf{c}_{\boldsymbol{x}^n} = \left(b^{(1)}_{\boldsymbol{x}^n}, b^{(2)}_{\boldsymbol{x}^n}, \ldots, b^{(\tilde{\ell}_{\boldsymbol{x}^n})}_{\boldsymbol{x}^n}\right).$$

Arithmetic Coding: Useful Observations for Decoding

- Notice

$$\begin{aligned} F_{\boldsymbol{x}^n} &= F_{x_1} + p_{x_1} F_{x_2} + p_{x_1} p_{x_2} F_{x_3} + \cdots + p_{x_1} p_{x_2} \cdots p_{x_{n-1}} F_{x_n} \\ &= F_{x_1} + p_{x_1} F_{\boldsymbol{x}_2^n}, \end{aligned}$$

  where $\boldsymbol{x}_2^n := (x_2, \ldots, x_n)$ and thus $F_{\boldsymbol{x}_2^n}$ does not depend on x_1 anymore.

- Let $\boldsymbol{x}^n = (x_1, \ldots, x_n)$ with $x_1 = i$ and $\boldsymbol{c}$ be the corresponding codeword. Also, let $F_{\boldsymbol{c}}$ be the decimal representation of $0.\boldsymbol{c}$. Then:

$$F_{x_1} = F_{x_1 11 \cdots 1} \leq F_{\boldsymbol{x}^n} \leq F_{x_1 m \cdots m} = F_{(x_1+1)1\cdots 1} - p_{x_1 m \cdots m} < F_{x_1+1},$$

  and

$$F_{\boldsymbol{c}} = \left(F_{\boldsymbol{x}^n} + 2^{-\tilde{\ell}_{\boldsymbol{x}^n}} \right)_{\text{cut after } \tilde{\ell}_{\boldsymbol{x}^n} \text{ bits}}.$$

- Combine these inequalities to obtain:

$$\begin{aligned} F_{\boldsymbol{c}} &\geq F_{\boldsymbol{x}^n} \geq F_{x_1} = F_i \\ 2^{-\tilde{\ell}_{\boldsymbol{x}^n}} + F_{\boldsymbol{c}} &\leq 2^{-\tilde{\ell}_{\boldsymbol{x}^n}} + F_{\boldsymbol{x}^n} + \frac{1}{2} p_{\boldsymbol{x}_2^n} \overset{\text{slide 5}}{\leq} F_{\boldsymbol{x}^n} + p_{\boldsymbol{x}^n} < F_{(x_1+1)1111} = F_{i+1} \end{aligned}$$

Arithmetic Coding for Memoryless Sources: Decoding Algorithm

1. Convert codeword $\mathbf{c}$ into a decimal number $F_{\boldsymbol{c}}$. Set $F_{\boldsymbol{c},1} = F_{\boldsymbol{c}}$.
2. For $t = 1, \ldots, n$ do:
   - Find $i \in \{1, \ldots, m\}$ such that $F_i \leq F_{\boldsymbol{c},t} < F_{i+1}$ and set $x_t = i$
   - Calculate
   $$F_{\boldsymbol{c},t+1} = \frac{F_{\boldsymbol{c},t} - F_i}{p_i}$$

Notice that since the quantization error of $\epsilon := F_{x^n} - F_{\boldsymbol{c}}$ is at most $2^{-\tilde{\ell}_{x^n}}$, one can show in a similar way that also

$$F_{x_t} \leq F_{c,t} \leq F_{x_t+1} \tag{3}$$

Decoding Algorithm of Multiple Codewords

What if multiple codewords are concatenated?
$\rightarrow$ algorithm still finds the correct codeword because adding more bits to the codeword can increase $F_{\boldsymbol{c}}$ by at most $2^{-\tilde{\ell}_{\boldsymbol{x}^n}}$. By the calculations in the previous slides: if $x_1 = i$, then

$$F_i \leq F_{\mathbf{c}} + 2^{-\tilde{\ell}_{\boldsymbol{x}^n}} < F_{i+1} \tag{4}$$

and

$$F_i \leq F_{\mathbf{c},\mathbf{t}} + \frac{2^{-\tilde{\ell}_{\boldsymbol{x}^n}}}{p_{\boldsymbol{x}_t^n}} < F_{i+1} \tag{5}$$

Arithmetic Coding for Sources with Memory: Encoding

- Consider the source $X_1, X_2, \ldots, X_n$ with pmf $p_{\boldsymbol{x}^{n-1}} = p_{x_1} Q(x_2|x_1) Q(x_3|x^2) \cdots Q(x_n|x^{n-1})$
- For the encoding we simply change the calculation of $F_{\boldsymbol{x}^n}$:

$$\begin{aligned} F_{\boldsymbol{x}^n} &:= \sum_{\boldsymbol{\xi}^n < \boldsymbol{x}^n} p_{\boldsymbol{\xi}^n} \\ &= \Pr[\boldsymbol{X}^n < \boldsymbol{x}^n] \\ &= \Pr\big[\boldsymbol{X}^{n-1} < \boldsymbol{x}^{n-1}\big] + \Pr\big[\boldsymbol{X}^{n-1} = \boldsymbol{x}^{n-1} \text{ and } X_n < x_n\big] \\ &= F_{\boldsymbol{x}^{n-1}} + p_{\boldsymbol{x}^{n-1}} \cdot \sum_{\xi < x_n} Q(\xi|\boldsymbol{x}^{n-1}) \\ &= F_{\boldsymbol{x}^{n-1}} + p_{\boldsymbol{x}^{n-1}} \cdot L_{x_n|\boldsymbol{x}^{n-1}} \end{aligned}$$

- Perform encoding as before but with new measure!

Arithmetic Coding for Sources with Memory: Decoding

Observe:

$$F_{\boldsymbol{x}^n} = F_{x_1} + p_{x_1} L_{x_2|x_1} + p_{x_1} Q(x_2|x_1) L_{x_3|\boldsymbol{x}^2} + \cdots + p_{x_1} Q(x_2|x_1) Q(x_3|\boldsymbol{x}^2) \cdots Q(x_{n-1}|\boldsymbol{x}^{n-2}) L_{x_n|\boldsymbol{x}^{n-1}}$$

Decoding algorithm:

1. Convert codeword $\mathbf{c}$ into a decimal number $F_{\boldsymbol{c}}$.
2. Find $i \in \{1, \ldots, m\}$ such that $F_i \leq F_{\boldsymbol{c}} < F_{i+1}$ and set $x_1 = i$
3. Calculate
$$F_{\boldsymbol{c},2} = \frac{F_{\boldsymbol{c}} - F_i}{p_i}$$
4. For $t = 2, \ldots, n$ do:
   - Find $i \in \{1, \ldots, m\}$ such that $L_{i|\boldsymbol{x}^{t-1}} \leq F_{\boldsymbol{c},t} < L_{i+1|\boldsymbol{x}^{t-1}}$ and set $x_t = i$
   - Calculate
$$F_{\boldsymbol{c},t+1} = \frac{F_{\boldsymbol{c},t} - L_{i|\boldsymbol{x}^{t-1}}}{Q(i|\boldsymbol{x}^{t-1})}$$

Window-Based Lempel-Ziv Algorithm; parameter W

Goal: Want to encode $X_1, \ldots, X_n$ without knowledge of their distribution

- Initialization $i = 1$
- while $i \leq n$
  - Look for longest length ℓ such that for some $j \in \{1, \ldots, W\}$:

$$(X_{i-j}, \ldots, X_{i-j+\ell-1}) = (X_i, \ldots, X_{i+\ell-1})$$

  - if $\ell > 0$: encode sequence with $F = 1$, $P = j$, $L = \ell$ and set $i = i + \ell$
  - otherwise if $\ell = 0$: encode next symbol with $F = 0$, $C = X_i$ and set $i = i + 1$

  end;

Concatenate the encodings for the various parsed substrings.

How to encode the various flags/pointers/...

- Flag F: a single bit
- New character C: binary representation in $\log |\mathcal{X}|$ bits
- Pointer P: binary representation of $P - 1$ in $\log W$ bits
- Length L: binary representation of $L - 1$ in $\log(\mathsf{L}_{\max})$ bits

If windowsize W very large or if maximum length $\mathsf{L}_{\max} = \infty$

$\rightarrow$ use a uniquely decodable unbounded integers encoding for P and L.

Uniquely Decodable Unbounded Integers Encoding

- Goal: want to encode an integer K which can be arbitrary large
- Encoding:
  - Let $m = \lceil \log(K) \rceil$
  - Encoding uses $2\lceil \log(K) \rceil + 1$ bits
  - Encode first m by using m 1's
  - Next bit is a 0
  - The last m bits are a binary representation of $K - 1$

Improvement

- Encode m using the unbounded integer encoding!
- Then, the encoding is:
  - First $\lceil \log(m) \rceil$ bits are all 1's, and the next bit is a 0
  - The next $\lceil \log(m) \rceil$ bits are a binary representation of $m - 1$
  - The last $\lceil \log(K) \rceil$ bits are a binary representation of $K - 1$
  - Encoding uses $\lceil \log(K) \rceil + 2\lceil \log(m) \rceil + 1 = \lceil \log(K) \rceil + 2\lceil \log\lceil \log(K) \rceil \rceil + 1$ bits

Performance of Lempel-Ziv Scheme

- Analysis in:
  A. Wyner and J. Ziv, "The sliding-window Lempel-Ziv Algorithm is asymptotically optimal", IEEE Transactions on Information Theory, June 1994, vol. 82, no. 6.

Theorem: [Scheme asymptotically optimal for most processes]

For every stationary ergodic process the window-based Lempel-Ziv coding needs only $H(\{X_k\}) + \epsilon(n)$ bits per source symbol to encode the sequence $X_1, \ldots, X_n$ where $\epsilon(n) \to 0$ as $n \to \infty$.

Intuition about the Proof

- Imagine
  - $W = \infty$
  - You look at supersymbols $Y_k = (X_{(k-1)N+1}, \ldots, X_{kN})$
- By ergodicity, when k very large and $P_{Y_k} > 0$ then Y_k has already appeared
- Therefore, only need to describe pointer P

  $\rightarrow$ use uniquely decodable unbounded integers encoding for P.

Average Expected Length per Source Symbol $\mathcal{L}_N$

$$\begin{aligned}\mathcal{L}_N &\approx \frac{1}{N}\mathsf{E}[\lceil\log(P)\rceil + 2\lceil\log\lceil\log(P)\rceil\rceil + 1] \\ &\approx \frac{1}{N}\mathsf{E}[\log(P)]\end{aligned}$$

when N very large, because $\mathsf{E}[\log\log(P)]/N \to 0$ as $N \to \infty$

For analysis we need two tools: Kac's Lemma & Jensen's Inequality!

Kac's Lemma for stationary ergodic processes $\{Y_k\}$

Lemma

Let

$$Q_y(i) \triangleq \Pr[Y_{-i} = y, Y_{-i+1}, \ldots, Y_{-1} \text{ all } \neq y | Y_0 = y] \,.$$

If $\Pr[Y_k = y] > 0$, *the expected time of the most recent occurrence of* y *after* 0 *is*

$$\sum_{i=1}^{\infty} i Q_y(i) = \frac{1}{\Pr[Y_k = y]}.$$

With this result and Jensen's Inequality

- Let $Y_k \triangleq (X_{(k-1)N+1}, X_{(k-1)N+2}, \ldots, X_{kN})$

$$
\begin{aligned}
N\mathcal{L}_N \approx \mathsf{E}[\log P] &= \sum_y \mathsf{Pr}[Y_1 = y]\, \mathsf{E}[\log P | Y_1 = y] \\
&\leq \sum_y \mathsf{Pr}[Y_1 = y] \log \mathsf{E}[P | Y_1 = y] \\
&= \sum_y \mathsf{Pr}[Y_1 = y] \log \frac{1}{\mathsf{Pr}[Y_1 = y]} \\
&= H(Y_1) = H(X_1, \ldots, X_N)
\end{aligned}
$$

- first equality: law of total expectation
- inequality: Jensen's inequality
- second equality: Kac's lemma
- third equality: definition of entropy

Finally

Thus, by letting $N \to \infty$,

$$\lim_{N\to\infty} \mathcal{L}_N \approx \lim_{N\to\infty} \frac{1}{N} H(X_1, \ldots, X_N) = H(\{X_k\}).$$

(Notice that $\lim_{N\to\infty} \mathcal{L}_N$ cannot be smaller than $H(\{X_k\})$, because otherwise it would violate the source coding theorem that holds when the distribution of the source is known.)